Universitatea Transilvania din Brașov

Universitatea Transilvania din Brașov
FACULTATEA DE INGINERIE ELECTRICĂ ȘI ȘTIINȚA CALCULATOARELOR

Departamentul : DEC
Programul de studii: SECI-SI

AILENEI Ioana
CHELEMEN Antonia
DEDU Madalina
MILCHIS Raul
PUSCASU Alexandru

PROIECT DE VERIFICARE

Brașov, 2022

Departamentul: DEC
Programul de studii: SECI-SI

AILENEI Ioana
CHELEMEN Antonia
DEDU Madalina
MILCHIS Raul
PUSCASU Alexandru

Inmultire matrici sistolice

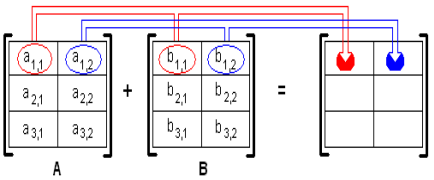
Brașov, 2022

FIȘA PROIECTULUI DE VERIFICARE

Universitatea Transilvania din Braşov	Anul universitar: 2023 - 2024
Facultatea de Inginerie Electrică şi Ştiința Calculatoarelor	

Membrii:
AILENEI Ioana - Model de referinta
CHELEMEN Antonia - Inginer de verificare
DEDU Madalina - Inginer de verificare
MILCHIS Raul - Arhitect
PUSCASU Alexandru - Inginer de proiectare

Coordonatori:
Alexandru Dinu

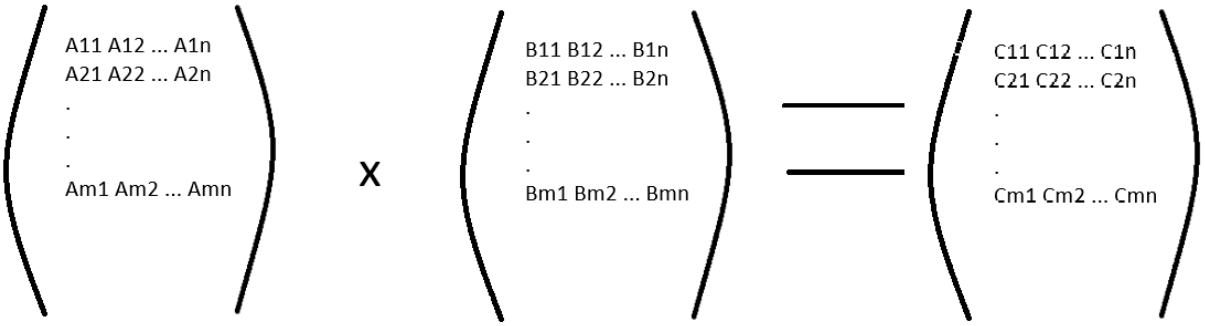


Sursa imaginii: https://ocw.cs.pub.ro/courses/programare/laboratoare/lab06

Titlul proiectului*: Inmultire matrici sistolice*

Sumarul funcționalităților DUT-ului:

1. AxB = C



2. Citește prin intermediul Req-Ack din memorie operanzii A și B pe 8 biți si calculează înmulțirea pe 16 biți.
3. Scrie C în memorie tot prin intermediul Req-Ack.
4. Protocolul APB configurează matricea prin accesarea unor regiștrii, mai exact 7 reg de 16 biți.

Sumarul scenariilor de verificare:

1. Verificam corectitudinea operatiilor efectuate
2. Verificam accesibilitatea registrilor APB
3. Verificam functionalitatea in caz de Reset

Istoricul versiunilor:

Versiune	Data finalizării	Numele editorului	Motiv
0.0	20.03.2021	Alexandru Dinu	Prima versiune de model
0.1	28.03.2022	Alexandru Dinu	Template-ul a fost actualizat

1.0	12.03.2024	Raul Milchis	Versiunea inițială

Cuprins

Cuprins
1 Structura proiectului 5
1.1 Locația proiectului 5
1.2 Ierarhia fișierelor 5
1.3 Modulul de top 5
2 Proiectul digital al circuitului 6
2.1 Porturile DUT-ului 6
2.2 Modul de funcționare al DUT-ului 6
2.3 Protocoalele folosite pentru comunicarea DUT-ului cu exteriorul 6
2.4 Informații legate de proiectarea DUT-ului 7
2.5 Alte informații 7
2.5.1 O secțiune 7
2.5.2 Altă secțiune a aceluiași subcapitol 7
3 Verificarea proiectului digital al circuitului 8
3.1 Scenariile de verificare 8
3.2 Implementarea modelului de referință 8
3.3 Secvențele de date 8
4 Interfețele 9
4.1 Interfețele utilizate și rolul acestora 9
4.2 Aserțiile 9
5 Mediul de verificare 10
5.1 Structura generală a mediului de verificare 10
5.2 Tranzacțiile 10
5.3 Testele 10
6 Rezultate 11
6.1 Rezultate 11
6.2 Concluzii generale 11

- ## LOCAȚIA PROIECTULUI

Git: https://github.com/alex2kameboss/Matrix-multiplier
EDA Playground: https://edaplayground.com/x/GXY3

1 Proiectul digital al circuitului

Diagrama bloc
Descrierea funcționalității generale a DUT-ului
Interfețele DUT-ului
Lista cu funcționalitățile DUT-ului
Schema de principiu
Forme de undă
Tabel cu regiștri

1.1 Diagrama bloc

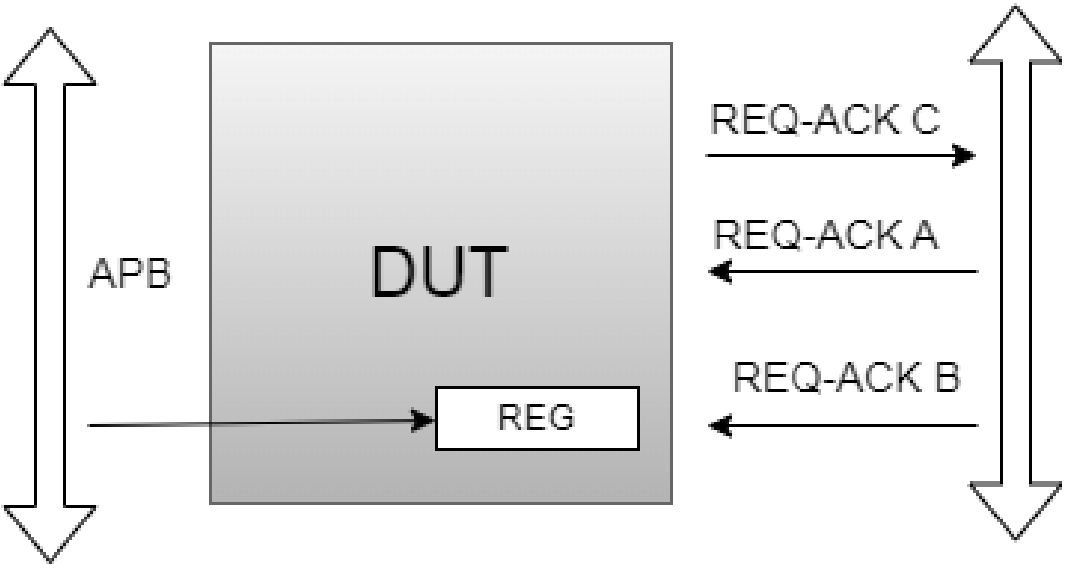


Figura 1. Exemplu de desen cu interfețele unui DUT cu funcția de a inmulti matrici. În acest caz interfața pentru configurare este APB, iar pentru comunicarea cu memoria avem 2 interfețe req_ack, pentru operanzi și pentru rezultat.

1.2 Descrierea funcționalității generale a DUT-ului

Sumarul funcționalităților DUT-ului:

1. A x B = C

$$\begin{pmatrix} A_{11} & A_{12} & \dots & A_{1n} \\ A_{21} & A_{22} & \dots & A_{2n} \\ \vdots & & & \\ A_{m1} & A_{m2} & \dots & A_{mn} \end{pmatrix} \times \begin{pmatrix} B_{11} & B_{12} & \dots & B_{1n} \\ B_{21} & B_{22} & \dots & B_{2n} \\ \vdots & & & \\ B_{m1} & B_{m2} & \dots & B_{mn} \end{pmatrix} = \begin{pmatrix} C_{11} & C_{12} & \dots & C_{1n} \\ C_{21} & C_{22} & \dots & C_{2n} \\ \vdots & & & \\ C_{m1} & C_{m2} & \dots & C_{mn} \end{pmatrix}$$

2. Citește prin intermediul Req-Ack din memorie operanzii A și B pe 8 biți si calculează înmulțirea pe 16 biți.
3. Scrie C în memorie tot prin intermediul Req-Ack.
4. Protocolul APB configurează matricea prin accesarea unor regiștrii, mai exact 7 reg de 16 biți.

1.3 Interfețele DUT-ului

Tabelul 1. INTERFAȚA DE SISTEM

NUME	PORT	DIMENSIUNE	FUNCTIONALITATE
clk	I	1	Clock
reset_n	I	1	Reset asincorn, active low

Tabelul 2. INTERFAȚA APB

NUME	PORT	DIMENSIUNE	FUNCTIONALITATE
psel	I	1	Slave selection signal
penable	I	1	Enable, indicates the second and subsequent APB transfer cycle
pwrite	I	1	Data Direction (1 = write, 0 = read)
paddr	I	3	APB Address for accessing registers
pwdata	I	16	Data to be written
prdata	O	16	Data to be read

pready	O	1	Ready signal to extend an APB transfer by the Slave

Tabelul 3. INTERFAȚA REQ-ACK

NUME	PORT	DIMENSIUNE	FUNCTIONALITATE
req	O	1	Request Data from/to Main Memory
addr	O	16	Request Address from/to Main Memory
data	I/O	256	Data
w_en	O	1	Data Direction (1=write, 0=read)
ack	I	1	Acknowledge signal

1.4 Lista cu funcționalitățile DUT-ului

Ft. 1 : AxB = C

Ft. 2: Citește prin intermediul Req-Ack din memorie A, B pe 8 biti si calculeaza inmultirea pe 16 biti

Ft. 3: Pe APB configurezi matricea, bancul de regiștrii este alcătuit din 7 reg de 16 biți :

- 1 reg care stochează adresa lui A
- 1 reg care stochează adresa lui B
- 1 reg care stochează adresa lui C
- 1 reg care are dimensiunea m
- 1 reg care are dimensiunea n
- 1 reg care are dimensiunea p
- 1 reg de stare (start, end)

Ft. 4: A de dimensiune m*n

B de dimensiune n*p

C de dimensiune m*p

Ft. 5: Scrie C in memorie tot prin intermediul Req-Ack

1.5 Schema de principiu

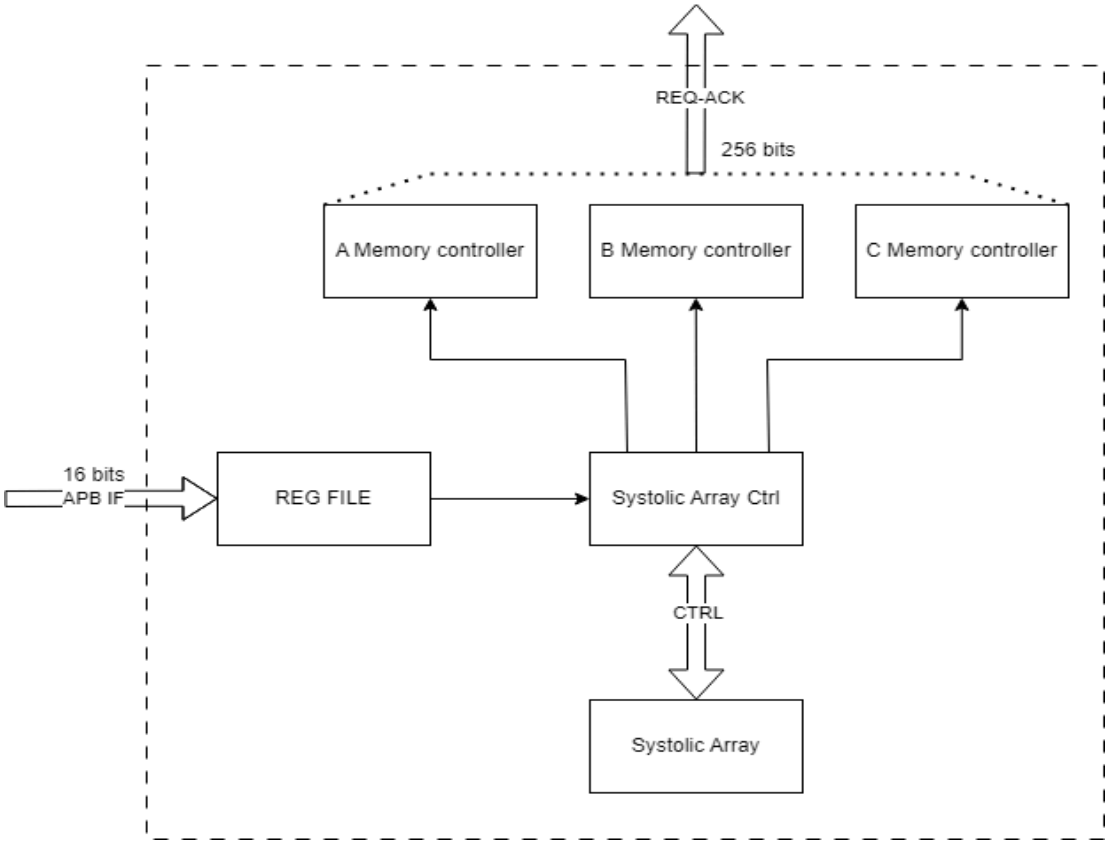


Figura 2. Schema bloc a DUT-ului

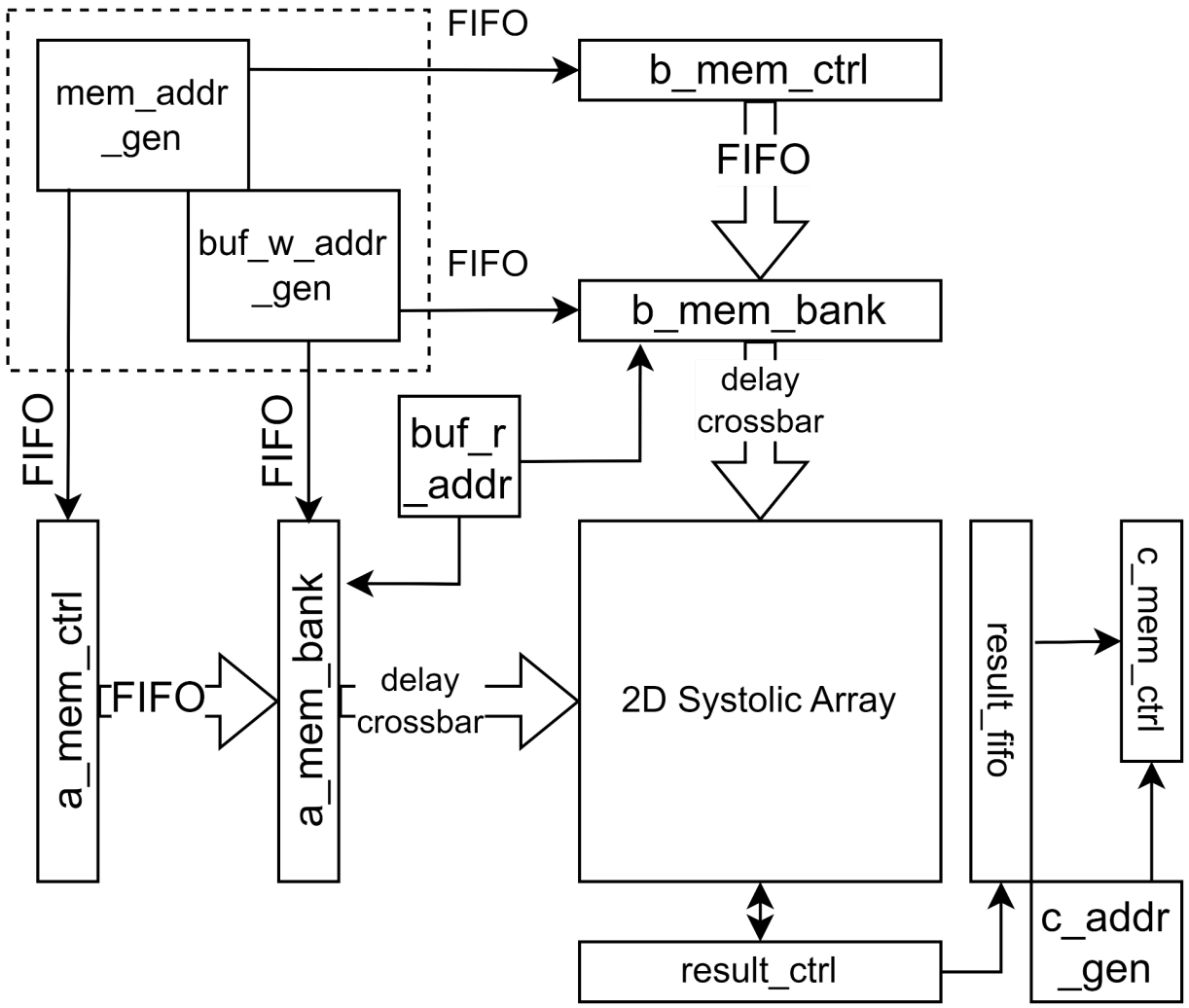


Figura 3. Schema RTL a DUT-ului

CONFIG - Seteaza parametrii m, n, p, addr

CTRL - Controleaza adresele operanzilor

2D Systolic Array - Multiplicatoare, Addere, PE (Processing Element)

FIFO - cozi folosite pentru sincronizare

addr generator - genereaza adrese pentru a pune datele in memorie (datele vin intr-o ordine stiuta)

mem bank - memorie

mem ctrl - trimite date independent de data-path

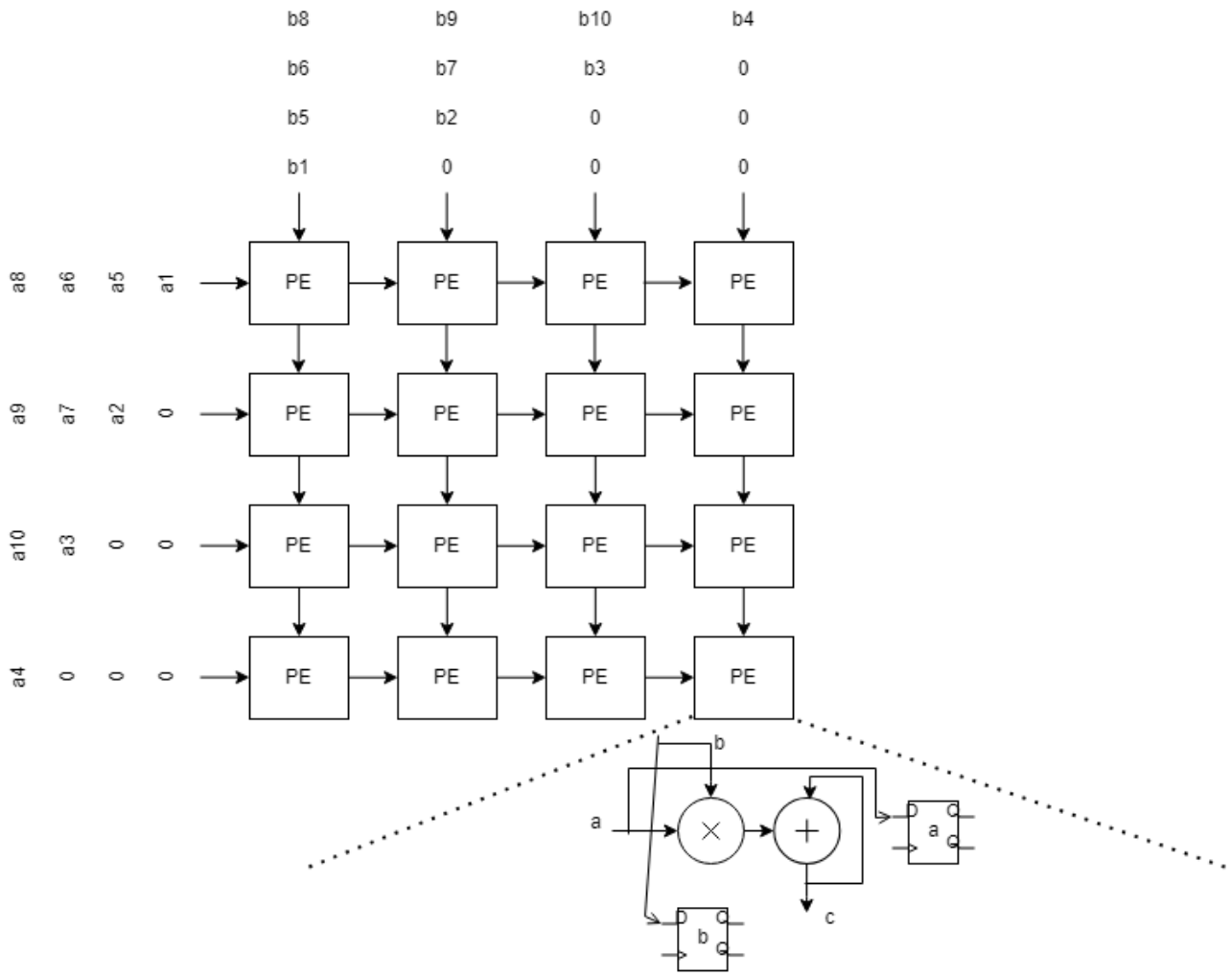


Figura 4. Schema microarhitecturala a 2D Systolic Array si reprezentarea unui PE (Processing Element)

- Enumerarea și descrierea parametrilor DUT-ului

DUT-ul are 2 parametri: ARRAY_HEIGHT și ARRAY_HEIGHT care specifica numărul de elemente de procesare a matricii sistolice.

- Probleme întâmpinate la proiectarea DUT-ului și cum au fost rezolvate acestea

Dimensionarea corectă a cozilor, pe partea de date de intrare componentele respective puteau sa aștepte până coada se eleibereaza, în cazul cozii de ieșire, datele vin regulat și nu pot aștepta, aici a trebuit sa prelungim cozile.

Proiectarea sistemului de preluare a datelor, datele sunt gata în ordinea deplasării diagonalei principale din colțul stânga sus pana în colțul stânga jos, nu este un pattern simplu și intuitiv de implemented in hardware.

Toate componentele care interacționează cu matricea sistolică trebuie sa fie proiectate sa fie rapide și fără latenta și intarzieri. Cand s-a pornit matricea sistolica, pana nu se termina ultima dată, ea nu se oprește, deci trebuie asigurat un flux constant de date.

- Detalii de implementare ale anumitor părți mai complicate ale DUT-ului.

Matricea A trebuie parcursă linie cu linie, iar matricea B trebuie parcursa coloana cu coloana, dar pentru A trebuie trimise la matricea sistolica datele de pe mai multe linii

consecutive simultan, la fel și pentru B mai multe coloane consecutive simultan. Pentru A aducem primele elemente de pe primele linii, și tot în maniera asta continuam pana la final. Asemanator procedam și pentru B. În memorii ținem datele exact cum trebuiesc ele trimise la matricea sistolică, deci înainte sa fie scrise, datele sunt ordonate.

La matricea sistolică trebuia sa ajunga întârziate cu 1 tact față de elementul anterior. Pentru aceasta am realizat un element triunghiular de intarziere, care ne ajută și cu trimiterea elementelor nule în tactele de început a matricii.

Pentru preluarea rezultatelor valide, am realizat un ”registru de deplasare” care are următoarea formulă:

r[i][j] = take_data, i = j = 0

r[i][j] = r[i - 1][j], i >= j

r[i][j] = r[i][j - 1], i < j

Pentru trimiterea datelor pe magistrală, datele fiind prea scurte, trebuiesc concatenate. Astfel, pentru fiecare ieșire există un registru de concatenare, care atunci cand are suficiente date le trimite în coadă pentru a fi trimise. Aceste registre se umplu înt-o ordine consecutiva, deci nu avem mai multe scrieri simultan.

Datele ajung în coadă, nu în ordine consecutivă, pentru această o componentă care generează adresele bune.

- Limitări în funcționarea DUT-ului (acestea pot să existe sau pot să nu existe; cu cât un DUT are mai puține limitări de funcționare, înseamnă că este mai de calitate, că este proiectat la un nivel mai bun; totuși, dacă respectivele limitări existe, acestea trebuie declarate). Exemple de astfel de limitări:
    - Portul către memorie are lățimea de 256 biti, iar adresele 16 biti, iar portul de configurare a registrilor de date 16 biti.
    - Dimensiunile trebuie sa fie multiplu de 32 și nu mai mari de 1024. Memoriile au adrese pe 16 biti, prin urmare dimensiunea maximă a matricii sa incapa pe 16 biți.

1.6 Forme de undă

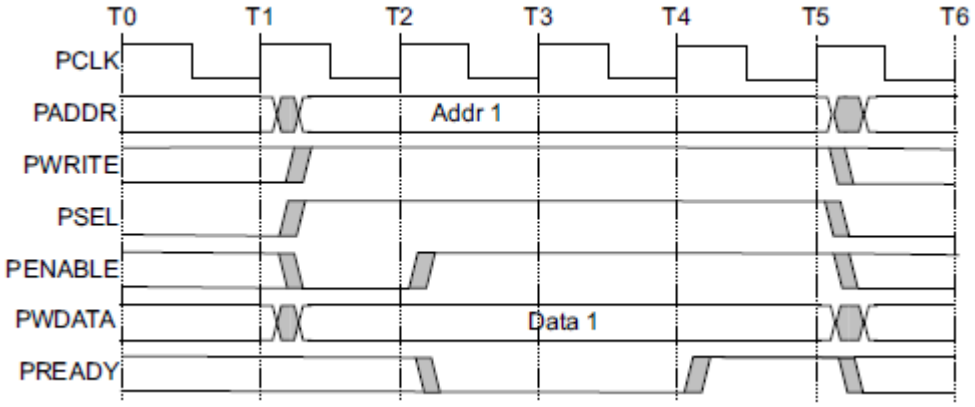


Figura 5. Formele de undă pentru APB write

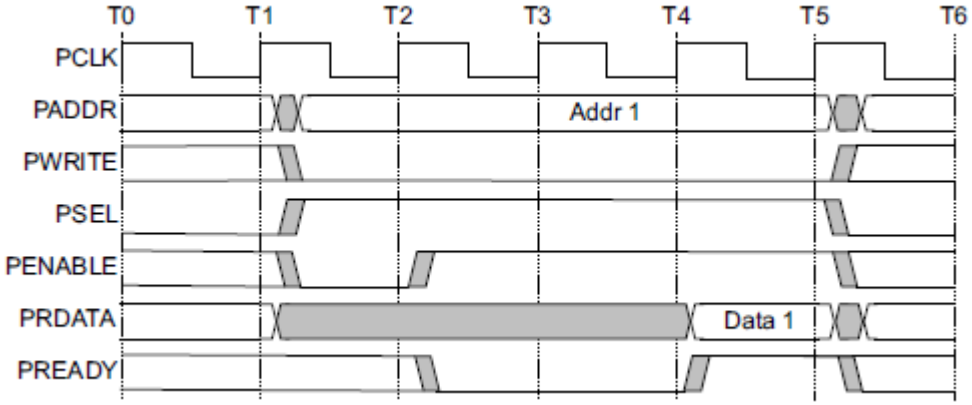


Figura 6. Formele de undă pentru APB read

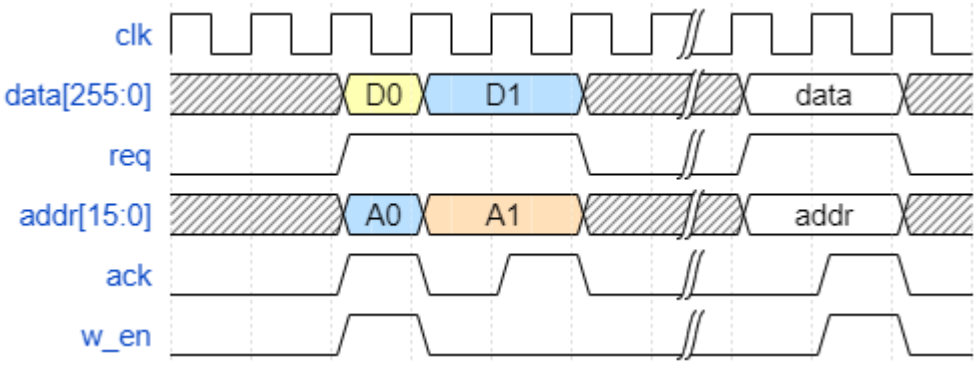
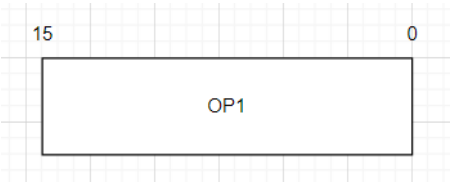
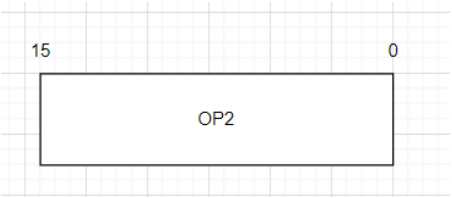
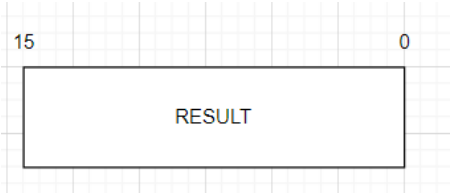
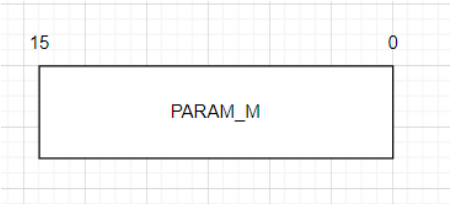
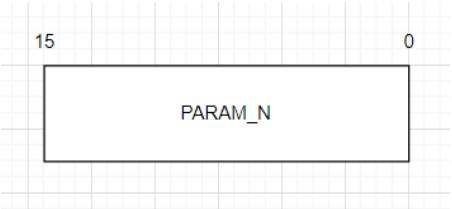
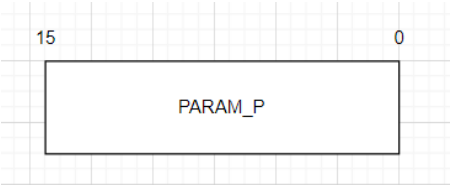
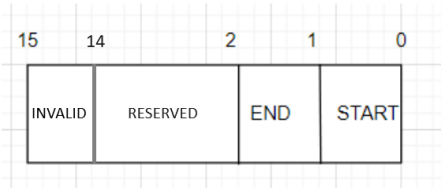


Figura 7. Formele de undă pentru request-acknowledge pentru read/write

1.7 Tabel cu regiștri

Tabelul 4. Tabel cu regiștrii care pot fi accesați de către DUT

Nume registru	Adresa de acces	Structura registrului	Descrierea câmpurilor registrului
OP1	0x00		OP1: Adresa matricei A (op1).
OP2	0x01		OP2: Adresa matricei B (op2).
RESULT	0x02		RESULT: Adresa matricei C (rezultatul).
PARAM_M	0x03		PARAM_M: M paramter (mA = M x N, mB = N x P, C = mM x P)
PARAM_N	0x04		PARAM_N: N paramter (mA = M x N, mB = N x P, mC = M x P)
PARAM_P	0x05		PARAM_P: P paramter (mA = M x N, mB = N x P, mC = M x P)
STATE	0x06		START[0]: start flag END[1]: end flag RESERVED[14:2]: empty space, default 0 can be X value. INVALID[15]: invalid input bit, raised when an invalid input is given.

2 Verificarea proiectului digital al circuitului

Arhitectura mediului de verificare

Lista cu elementele de verificat

Elementele de acoperire funcțională

Testele

Rezultatele obținute

2.1 Arhitectura mediului de verificare

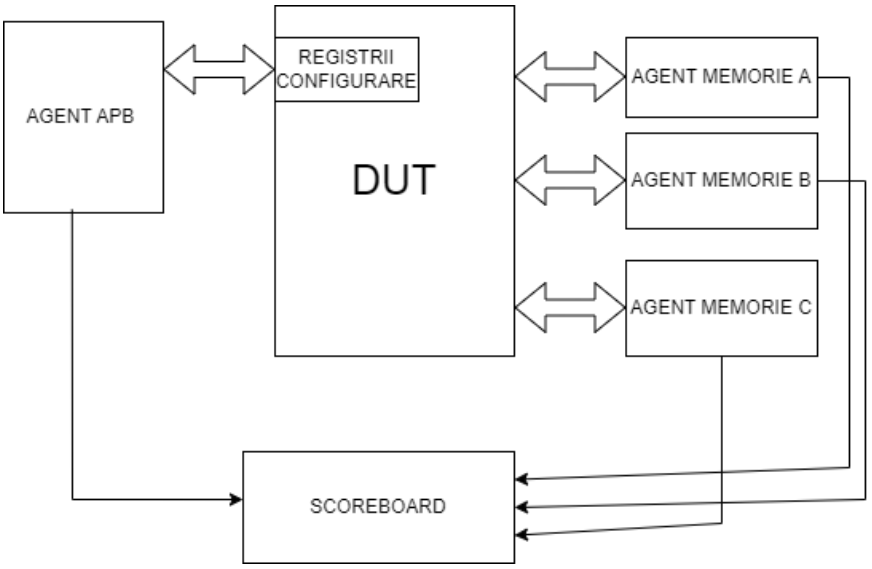


Figura 8. Imaginea de ansamblu a mediului de verificare

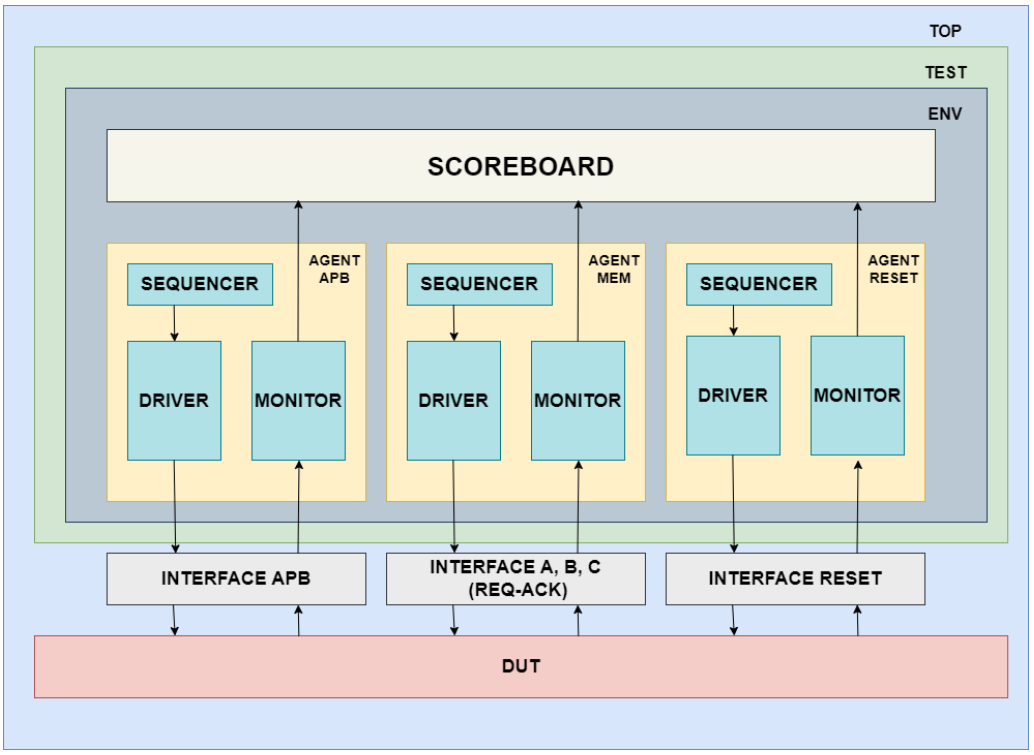


Figura 9: Mediu de verificare: Testbench si test environment

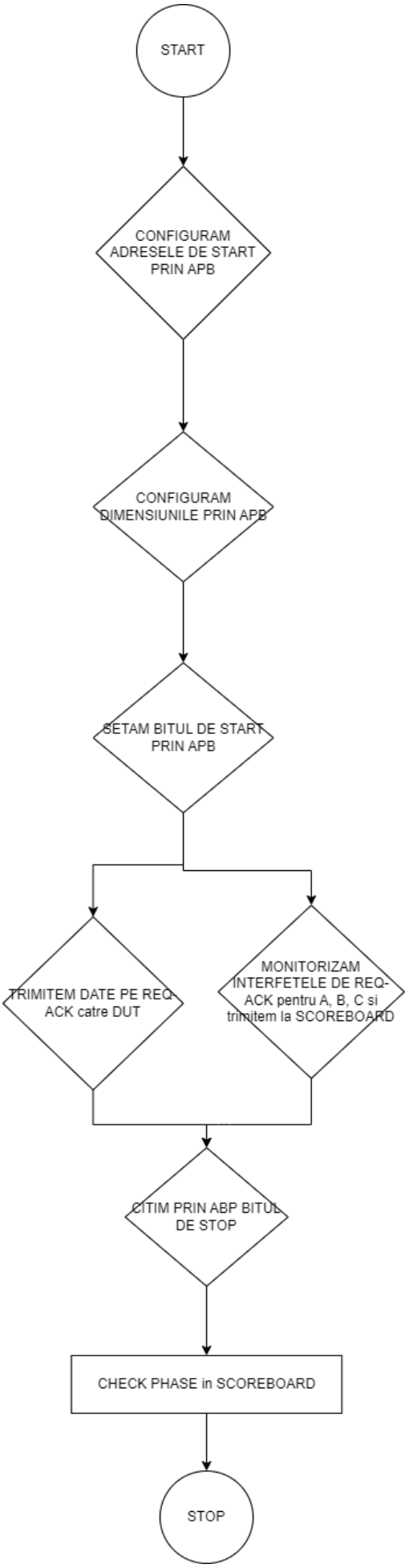


Figura 10. Schemă logică.

2.1.1 Driver pentru interfața REQ-ACK

Driverul este o componentă activă a mediului de verificare care transmite pachete pe interfață pentru comunicarea cu DUT. Pentru memorie am creat 3 drivere în care am implementat protocolul Request-Acknowledge. În primul driver, cel prin care se transmit datele pentru matricea a, am implementat protocolul Request-Acknowledge. Se așteaptă semnalul de request, care vine de la DUT, pentru a pune semnalul de acknowledge în 1 și a pune datele pe interfață. Pentru driverul care transmite datele pentru matricea b se implementează la fel protocolul și se pun datele pe interfață. Driverul pentru citirea matricii c transmite doar semnalul de acknowledge pentru a anunța DUT ca se poate efectua tranzacția și ca aceasta s-a terminat.

2.1.2 Driver APB

Driverul interacționează cu DUT-ul. El trimite tranzacții către DUT folosind interfața. Aceste tranzacții pot include citiri (read) și scrieri (write) către registrii respectand specificatiile protocolului APB. Penable se asertează la un tact după psel. Daca avem psel se transmite: adresa, data și pwrite (tipul de tranzactie). Dacă avem scriere sa scrie pwdata cu valoarea din item, dacă avem citire se va lua valoarea de pe prdata.

2.1.3 Scoreboard / modul referinta

Scoreboard-ul funcționează ca o entitate de control și verificare în cadrul verificării funcționale a modulului de înmulțire a două matrice. Acesta utilizează patru porturi pentru comunicarea cu monitoarele. Interfețele de memorie primesc adrese și date, pe care le adaugă într-un array asociativ adresa: date. Interfața APB trimite pachetul de configurare a DUT-ului, (dimensiunile matricelor, adresele, statusul operației (START/STOP). După ce toate datele sunt primite, se construiesc matricile A, B și C, folosind informațiile primite și ordonând datele în funcție de adrese. Se construiesc, de asemenea, matricile bazandu-ne pe dimensiunile primite (expA, expB, expC). Scoreboard-ul verifică apoi corectitudinea operației de înmulțire, comparând rezultatul așteptat (AxB) cu rezultatul efectiv (C). În plus, se efectuează o verificare a corectitudinii adreselor pentru a asigura că accesările din memorie sunt efectuate conform configurației primite.

2.1.4 Monitor APB

Monitorul este o componenta pasiva folosita pentru a captura semnalele DUT-ului prin intermediul interfeței virtuale si pentru a le traduce in formatul folosit de secvente. Aceste elemente de secventa sunt transmise catre alte componente precum scoreboardul. Monitorul transmite un port de analiza TLM (Tranzaction Level Mondeling) pentru a transmite tranzactiile.

2.1.5 Monitor pentru interfața REQ-ACK

Monitorul este o componentă UVM care este folosită pentru a monitoriza și pentru a captura semnalele de pe interfață pentru a le transmite prin componenta de tranzacție altor componente. În monitorul pentru matricea a se așteaptă ca datele să fie valide, să avem semnalele de request și acknowledge, și se iau datele de pe interfață pentru a fi transmise mai departe. Celelalte monitoare, pentru matricile b si c sunt implementate la fel ca cel pentru matricea a.

2.1.6 Tranzactie pentru interfața APB

O clasa de tip tranzactie în UVM este un mecanism flexibil si modular pentru a reprezenta și manipula datele și semnale în cadrul mediului de testare și pentru a facilita interacțiunea între diferitele componente ale testbench-ului. Pentru a realiza o tranzactie APB trebuie configura bitul de start al registrului 6. Adresele operanzilor a,b,c se vor scrie la registrii 0,1 respectiv 2. Parametrii m, n, p vor fi scrisi la registrii 3, 4, 5. Configurarea registrilor este descrisă în capitolul 1.7 Tabel cu regiștrii.

2.1.7 Tranzactie pentru interfața REQ-ACK

O clasa de tip tranzactie în UVM este un mecanism flexibil si modular pentru a reprezenta și manipula datele și semnale în cadrul mediului de testare și pentru a facilita interacțiunea între diferitele componente ale testbench-ului. Am folosit un singur fișier tranzacție pentru toate 3 matricile deoarece este aceeași structură, conține date, adrese, write enable și delay.

2.1.8 Agent pentru interfața REQ-ACK

Un agent încapsuleaza Seguencer-ul, Driver-ul si Monitorul într-o singura entitate conectand componentele cu ajutorul interfeței virtuale.

2.1.9 Agent pentru interfața APB

Un agent incapsuleaza Seguencer-ul, Driver-ul si Monitorul într-o singura entitate conectand componentele cu ajutorul interfeței virtuale.

2.1.10 Colector de coverage pentru interfața REQ-ACK

2.1.11 Colector de coverage pentru scoreboard / model referinta

Colectorul de acoperire specific pentru multiplicatorul de matrice este adaptat pentru a evalua eficiența și acuratețea testelor în cadrul verificării funcționale a acestui modul specific. Acesta monitorizează și înregistrează aspecte critice legate de operațiile de înmulțire a matricelor, precum acoperirea liniilor și coloanelor din matrice, acoperirea combinațiilor de elemente, precum și acoperirea diferitelor scenarii de încărcare și decădere a datelor. De exemplu: verificam ca am testat matrici de dimensiuni NxN, NxN, NxN cât și de dimensiuni MxN, NxP, MxP, ca am testat diferite adrese, că am testat cazuri critice cum ar fi înmulțirea cu 0 etc. (detaliat 2.3)

2.2 Lista cu elementele de verificat

Checker	Descriere
multiply_check	Verifica daca AxB=C
data_integrity_check	Verifica daca datele primite la iesire sunt conform cu cele asteptate (expC == actualC).
addr_check	Verifica daca adresa de inceput de pe REQ-ACK este identica cu cea de pe APB A, B, C
status_check	Verifica daca operatia de inmultire se desfasoara conform celor 2 biti de status din registrul de status (START/STOP)
config_check	Verificam ca DUT-ul functioneaza conform configuratiei de pe APB

Checker APB IF	Descriere
idle_state	Daca paddr s-a schimbat inseamna ca transferul s-a terminat (la aceiasi adresa) sau nu este un transfer valid
setup_state	Verifica daca penable s-a ridicat in urmatorul tact dupa ce

	psel a fost ridicat
access_wait_state	Verifica daca in timpul unui acces cu timpi de asteptare psel si penable raman HIGH si doar pready face toggle.
pr_generic_stable(signal)	Verifica daca semnalele sunt stabile in timpul unui transfer
pr_generic_not_unknown (signal)	Checker pentru a asigura ca semnalele nu sunt in X (Warning message)
pwdata_in_read_transfer	Verifica daca pwdata se schimba in timpul unui transfer de citire
prdata_in_wr_transfer	Verifica daca prdata se schimba in timpul unui transfer de scriere
penable_in_transfer	Penable trebuie sa ramana HIGH pe tot timpul transferului.
psel_stable_in_transfer	Psel trebuie sa fie stabil (HIGH) in timpul transferului atunci cand penable este stabil (HIGH) in tot timpul unui transfer.

Checker REQ-ACK IF (MEM IF)	**Descriere**
pr_generic_not_unknown (signal)	Checker pentru a asigura ca semnalele nu sunt in X (Warning message)
ack_pulse	Verifica daca semnalul acknowledge este un puls.
req_without_ack	Verifica situatia in care ai un request fara acknowledge venit, request-ul pica fara acknowledge
ack_without_req	Verifica daca acknowledge se activeaza fara un request activ.
ack_after_one_req_cc	Verifica ca acknowledge sa nu vina in acelasi tact cand request-ul s-a ridicat.

2.3 Elementele de acoperire funcțională

Eveniment: cover_trans_done_ev: emis la finalul unei tranzactii burst	
Cover Point	**Descriere**
dim_cp	Diverse Dimensiuni ale Matricei : min 32 max 1024, mzultipli de 32 cat si gresite (0?)
addr_cp	Tot range-ul de adrese a fost acoperit. Adresele sunt pe 16

	de biti, se considera adresa minima, maxima, iar restul range-ului este impartit in 16 de parti egale, cat si out of range
equal_dim_cp	S-au facut operatia de inmultire cu matrici de aceiasi dimensiune N (NxN, NxN, NxN) si cum alte dimensiuni (MxN, NxP, MxP), cat si gresite
status_cp	toggle START, STOP
ack_delay_cp	ack s-a setat dupa 1 tact, 2 tacte, [3:10] tacte, >10tacte
req_delay_cp	req s-a dat dupa 1 tact, 2 tacte, [3:10] tacte, >10tacte
start_stop_delay	cat a durat operatia de inmultire
data_cp	numere întregi, numere reale, valori negative, zero și valori maxime
transfer_cp	o config-o operatie, o config- n operatii, n config-n operatii
checkers_cp	verifica ca toate checkere au trecut(PASS) macar o data

2.4 Testele

Se va realiza un tabel cu testele implementate

Tabelul 5. Lista testelor implementate pentru verificarea DUT-ului

Numele testului	Descrierea testului
config_test	se realizeaza o configurare pe APB
one_op_test	configuram pe APB dupa care facem o operatie cu dimesiuni ale matricilor egale(64x64)
two_op_test	configuram pe APB dupa care facem o operatie, asteptam done si facem inca o operatie (start repetat)
two_config_test	configuram pe APB dupa care facem o operatie, asteptam done si configuram pe APB din nou, dupa care facem inca o operatie
random_test	configuram random pe APB dupa care facem n operatii cu date random si delay random
wrong_dim_config_test	configuram dimensiunile gresit ca sa vedem ca dut-ul nu executa operatia
wrong_addres_config_test	configuram adresele gresit (in afara spatiului de memorie) ca sa vedem ca dut-ul nu executa operatia

wrong status_addres_config	nu configuram start in registrul de stare ca sa vedem ca dut-ul nu executa operatia
random_with_wrong_and _correct_config	configuram random pe APB (configuratie corecta sau gresita de mai multe ori) dupa care facem operatii cu date random si delay random
test_overflow	

2.5 Limitari

L1. Datele de req ack trebuie sa fie multiplu de 32
L2. Configuratia nu poate fi schimbata in timpul operatiei